

Práctica de refuerzo — Unidad 2

80 ejercicios de programación PL/SQL sobre Juanita's Market

Identificación

Dato	Información
Nombre	
Sección	
Fecha	
Curso	Base de Datos
Puntaje referencial	80 puntos

Propósito y estructura

Esta práctica prepara para los ocho requerimientos de programación PL/SQL de la ES2. Todos los ejercicios utilizan el modelo relacional de **Juanita's Market** y el prefijo de trabajo **GIGH_**, que puede reemplazarse por las iniciales del estudiante.

Cada sección corresponde a un requerimiento y contiene diez ejercicios:

- **Las primeras seis preguntas:** afianzamiento progresivo de la habilidad.
- **Las últimas cuatro preguntas:** desafíos que combinan objetos, validaciones, concurrencia, transacciones y excepciones.

Una solución equivalente es válida si respeta el modelo, los nombres solicitados y el comportamiento indicado.

Instrucciones

1. Resuelva cada ejercicio en Oracle PL/SQL y escriba antes de cada respuesta un comentario como `-- Pregunta 1.`
2. Utilice el prefijo **GIGH_** en tablas, restricciones, secuencias, triggers, funciones y procedimientos.
3. Ejecute los bloques PL/SQL con `/` después de cada definición.
4. Pruebe primero en un esquema de laboratorio y consulte los objetos compilados con `SHOW ERRORS`.
5. Controle las transacciones con `COMMIT` y `ROLLBACK`; no deje datos de prueba sin documentar.
6. Use `:NEW` y `:OLD` según el evento del trigger. Un trigger no debe ejecutar `COMMIT`, `ROLLBACK` ni `SAVEPOINT`.
7. Maneje las excepciones específicas antes de `WHEN OTHERS` y conserve el diagnóstico mediante `SQLCODE` y `SQLERRM`.
8. En cada procedimiento diferencie los parámetros (`_P`) de las columnas y aplique control de concurrencia cuando el requerimiento lo indique.

Caso y modelo relacional

Juanita desea ampliar su peluquería con un minimarket. El sistema debe administrar clientes, productos, ventas, detalles de venta y un registro de operaciones. El diseño contiene las tablas **Cliente**, **Productos**, **Venta**, **Detalle_Venta** y **Log_Ventas**.

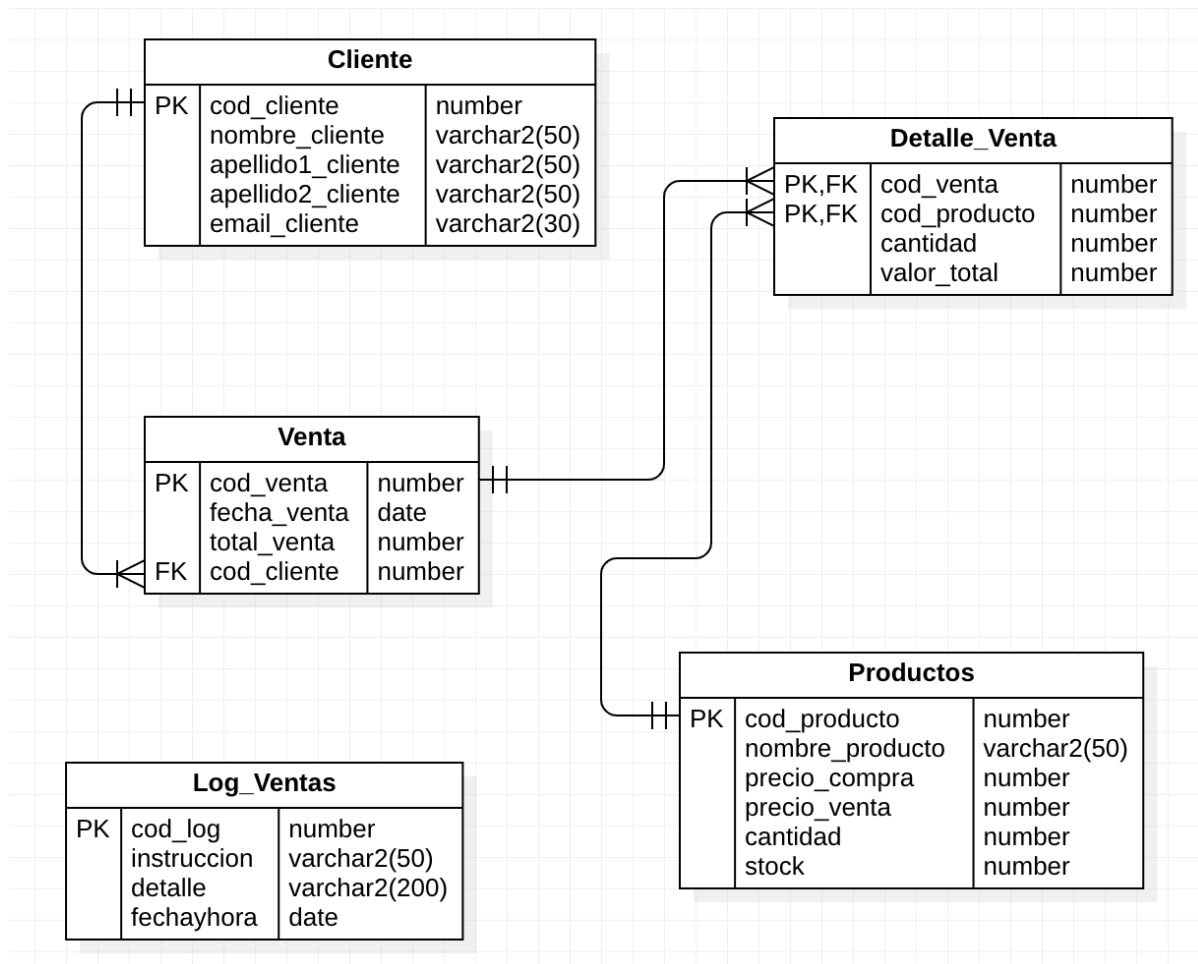


Figura 1: Modelo relacional de Juanita's Market

En Oracle utilice los nombres físicos GIGH_CLIENTE, GIGH_PRODUCTOS, GIGH_VENTA, GIGH_DETALLE_VENTA y GIGH_LOG_VENTAS. Las claves primarias son COD_CLIENTE, COD_PRODUCTO, COD_VENTA y COD_LOG. La tabla de detalle tiene clave primaria compuesta por COD_VENTA y COD_PRODUCTO.

Requerimientos de la ES2

1. Crear el modelo en SQL, incluyendo `CREATE TABLE`, `DROP TABLE` ordenados, claves primarias y foráneas.
2. Crear triggers que generen las claves primarias autoincrementales de clientes y productos.
3. Crear triggers que conviertan a mayúsculas los campos alfanuméricos de clientes y productos.
4. Crear el trigger `RECTIFICADOR` para actualizar los detalles cuando cambie el código de una venta.
5. Crear una función para correos con formato `NOMBRE.APELLIDO@JUANISMARKET.CL` y sufijo numérico para duplicados.
6. Crear un procedimiento integral de productos con opciones R, U y D.
7. Crear un procedimiento de inserción para clientes.
8. Crear una secuencia para la clave primaria de `LOG_VENTAS`.

Sección 1 — Modelo relacional y orden de objetos

Habilidad central: modelo físico, restricciones, dependencias y limpieza idempotente.

Pregunta 1 — Afianzamiento

Escriba los `CREATE TABLE` de `GIGH_CLIENTE` y `GIGH_PRODUCTOS` con sus columnas, tipos, claves primarias y restricciones de obligatoriedad.

Respuesta PL/SQL:

Pregunta 2 — Afianzamiento

Complete las tablas `GIGH_VENTA`, `GIGH_DETALLE_VENTA` y `GIGH_LOG_VENTAS`. Incluya la clave primaria compuesta del detalle.

Respuesta PL/SQL:

Pregunta 3 — Afianzamiento

Agregue las claves foráneas entre cliente-venta, venta-detalle y producto-detalle. Use nombres de restricciones con prefijo GIGH_.

Respuesta PL/SQL:

Pregunta 4 — Afianzamiento

Ordene los `DROP TABLE ... CASCADE CONSTRAINTS` para eliminar el modelo completo sin errores de dependencia.

Respuesta PL/SQL:

Pregunta 5 — Afianzamiento

Construya un script idempotente que capture ORA-00942 al intentar eliminar una tabla inexistente y relance cualquier otro error.

Respuesta PL/SQL:

Pregunta 6 — Afianzamiento

Inserte un registro válido en cada tabla y compruebe con consultas que las claves y relaciones quedaron creadas.

Respuesta PL/SQL:

Pregunta 7 — Desafío

Desafío: agregue restricciones **CHECK** para precios, cantidades, stock y totales, y demuestre con una inserción inválida qué excepción produce Oracle.

Respuesta PL/SQL:

Pregunta 8 — Desafío

Desafío: diseñe una versión del modelo que permita eliminar un cliente solo si no tiene ventas, explicando el efecto de la clave foránea.

Respuesta PL/SQL:

Pregunta 9 — Desafío

Desafío: escriba un bloque que consulte `USER_CONSTRAINTS` y `USER_CONS_COLUMNS` para listar las restricciones del modelo `GIGH_`.

Respuesta PL/SQL:

Pregunta 10 — Desafío

Desafío: integre limpieza, creación, carga mínima y consultas de verificación en un único script ejecutable desde un esquema vacío.

Respuesta PL/SQL:

Sección 2 — Secuencias y triggers autoincrementales

Habilidad central: secuencias, :NEW, claves automáticas y pruebas de generación.

Pregunta 11 — Afianzamiento

Cree GIGH_SEQ_CLIENTE con START WITH 1, incremento unitario y NOCYCLE.

Respuesta PL/SQL:

Pregunta 12 — Afianzamiento

Cree GIGH_SEQ_PRODUCTO con límites razonables para el crecimiento del minimarket.

Respuesta PL/SQL:

Pregunta 13 — Afianzamiento

Cree GIGH_SEQ_LOG_VENTA y documente por qué una secuencia no garantiza números consecutivos.

Respuesta PL/SQL:

Pregunta 14 — Afianzamiento

Implemente GIGH_CLIENTE_AUTOINCREMENTAL para asignar NEXTVAL cuando :NEW.COD_CLIENTE sea nulo.

Respuesta PL/SQL:

Pregunta 15 — Afianzamiento

Implemente GIGH_PRODUCTO_AUTOINCREMENTAL y pruebe dos inserciones sin indicar código.

Respuesta PL/SQL:

Pregunta 16 — Afianzamiento

Pruebe qué ocurre cuando se entrega un código explícito y explique por qué el trigger no debe reemplazarlo.

Respuesta PL/SQL:

Pregunta 17 — Desafío

Desafío: cree un trigger autoincremental para `GIGH_LOG_VENTAS` y registre tres operaciones usando la secuencia.

Respuesta PL/SQL:

Pregunta 18 — Desafío

Desafío: simule dos sesiones insertando clientes y explique por qué una secuencia evita que ambas obtengan el mismo código.

Respuesta PL/SQL:

Pregunta 19 — Desafío

Desafío: provoque un ROLLBACK después de consumir NEXTVAL y compruebe que el número puede quedar con un salto.

Respuesta PL/SQL:

Pregunta 20 — Desafío

Desafío: escriba un bloque que consulte `USER_SEQUENCES` y valide inicio, incremento, máximo, ciclo y caché de las secuencias.

Respuesta PL/SQL:

Sección 3 — Triggers de mayúsculas

Habilidad central: BEFORE, INSERT, UPDATE, UPPER, TRIM y valores nulos.

Pregunta 21 — Afianzamiento

Cree GIGH_CLIENTE_MAYUSCULAS para convertir nombre y apellidos a mayúsculas antes de insertar.

Respuesta PL/SQL:

Pregunta 22 — Afianzamiento

Amplíe el trigger para normalizar también EMAIL_CLIENTE y eliminar espacios laterales.

Respuesta PL/SQL:

Pregunta 23 — Afianzamiento

Cree GIGH_PRODUCTO_MAYUSCULAS para normalizar NOMBRE_PRODUCTO en inserciones.

Respuesta PL/SQL:

Pregunta 24 — Afianzamiento

Modifique ambos triggers para que funcionen en INSERT OR UPDATE.

Respuesta PL/SQL:

Pregunta 25 — Afianzamiento

Inserte nombres en minúsculas con espacios y compruebe el resultado almacenado.

Respuesta PL/SQL:

Pregunta 26 — Afianzamiento

Actualice un producto y un cliente con texto mixto y valide nuevamente la normalización.

Respuesta PL/SQL:

Pregunta 27 — Desafío

Desafío: maneje valores nulos sin convertirlos en una cadena vacía cuando la columna sea opcional.

Respuesta PL/SQL:

Pregunta 28 — Desafío

Desafío: agregue una validación que rechace nombres vacíos con `RAISE_APPLICATION_ERROR` y un código entre -20030 y -20039.

Respuesta PL/SQL:

Pregunta 29 — Desafío

Desafío: normalice correos con `LOWER` en vez de `UPPER` y explique qué decisión es más adecuada para la presentación y la unicidad.

Respuesta PL/SQL:

Pregunta 30 — Desafío

Desafío: compruebe que los triggers no modifican las columnas numéricas de precios, cantidad o stock.

Respuesta PL/SQL:

Sección 4 — Trigger RECTIFICADOR

Habilidad central: actualización de claves, :OLD, :NEW y detalle de ventas.

Pregunta 31 — Afianzamiento

Cree una venta y un detalle asociado y consulte ambas tablas antes de modificar el código de venta.

Respuesta PL/SQL:

Pregunta 32 — Afianzamiento

Implemente GIGH_RECTIFICADOR como trigger BEFORE UPDATE OF COD_VENTA sobre GIGH_VENTA.

Respuesta PL/SQL:

Pregunta 33 — Afianzamiento

Actualice una venta de código 5 a 6 y compruebe que todos sus detalles adoptaron el código nuevo.

Respuesta PL/SQL:

Pregunta 34 — Afianzamiento

Pruebe una venta con dos productos y verifique que se actualizan varias filas del detalle.

Respuesta PL/SQL:

Pregunta 35 — Afianzamiento

Compruebe que un UPDATE sobre FECHA_VENTA no dispara la lógica de rectificación.

Respuesta PL/SQL:

Pregunta 36 — Afianzamiento

Muestre con una consulta la diferencia entre :OLD.COD_VENTA y :NEW.COD_VENTA.

Respuesta PL/SQL:

Pregunta 37 — Desafío

Desafío: controle que el nuevo código no exista antes de modificar la venta y genere un error personalizado.

Respuesta PL/SQL:

Pregunta 38 — Desafío

Desafío: maneje la actualización de una venta sin detalles y registre en `DBMS_OUTPUT` cuántas filas fueron afectadas.

Respuesta PL/SQL:

Pregunta 39 — Desafío

Desafío: explique cómo la actualización del detalle evita una violación de la clave foránea durante el cambio de código.

Respuesta PL/SQL:

Pregunta 40 — Desafío

Desafío: agregue una auditoría que registre en `GIGH_LOG_VENTAS` el código anterior, el nuevo código y la fecha del cambio.

Respuesta PL/SQL:

Sección 5 — Función de correos

Habilidad central: funciones, `SELECT INTO`, duplicados, sufijos y excepciones.

Pregunta 41 — Afianzamiento

Declare la estructura de `GIGH_GENERA_EMAIL` con parámetros de nombre y primer apellido y retorno `VARCHAR2`.

Respuesta PL/SQL:

Pregunta 42 — Afianzamiento

Genere el correo base con formato `NOMBRE.APELLIDO@JUANISMARKET.CL`.

Respuesta PL/SQL:

Pregunta 43 — Afianzamiento

Aplique TRIM y una normalización consistente de mayúsculas o minúsculas.

Respuesta PL/SQL:

Pregunta 44 — Afianzamiento

Consulte GIGH_CLIENTE para detectar si el correo base ya existe.

Respuesta PL/SQL:

Pregunta 45 — Afianzamiento

Agregue el sufijo 1 antes de la arroba cuando exista el correo base.

Respuesta PL/SQL:

Pregunta 46 — Afianzamiento

Extienda la lógica para producir los sufijos 2, 3 y siguientes sin duplicar correos.

Respuesta PL/SQL:

Pregunta 47 — Desafío

Desafío: controle nombres o apellidos nulos con una excepción propia y `RAISE_APPLICATION_ERROR`.

Respuesta PL/SQL:

Pregunta 48 — Desafío

Desafío: elimine tildes y espacios internos de nombres y apellidos antes de construir el correo.

Respuesta PL/SQL:

Pregunta 49 — Desafío

Desafío: pruebe la función con cuatro clientes repetidos y muestre los cuatro correos generados.

Respuesta PL/SQL:

Pregunta 50 — Desafío

Desafío: explique la condición de carrera entre dos sesiones y proponga un bloqueo o una restricción **UNIQUE** para garantizar la integridad.

Respuesta PL/SQL:

Sección 6 — Procedimiento integral de productos R/U/D

Habilidad central: parámetros, DML, opciones, concurrencia, transacciones y log.

Pregunta 51 — Afianzamiento

Declare GIGH_PRODUCTOS_RUD con OPCION_P y parámetros para todos los datos del producto.

Respuesta PL/SQL:

Pregunta 52 — Afianzamiento

Implemente la opción R para insertar un producto usando la clave automática cuando corresponda.

Respuesta PL/SQL:

Pregunta 53 — Afianzamiento

Implemente la opción U para modificar precio de compra, precio de venta y stock mediante el código.

Respuesta PL/SQL:

Pregunta 54 — Afianzamiento

Implemente la opción D para eliminar un producto mediante el código.

Respuesta PL/SQL:

Pregunta 55 — Afianzamiento

Agregue IF o ELSIF para rechazar opciones distintas de R, U y D.

Respuesta PL/SQL:

Pregunta 56 — Afianzamiento

Use `SQL%ROWCOUNT` para informar cuando una actualización o eliminación no encontró el producto.

Respuesta PL/SQL:

Pregunta 57 — Desafío

Desafío: agregue `LOCK TABLE`, `COMMIT`, `ROLLBACK`, `DUP_VAL_ON_INDEX`, `VALUE_ERROR` y `WHEN OTHERS`.

Respuesta PL/SQL:

Pregunta 58 — Desafío

Desafío: registre cada operación en `GIGH_LOG_VENTAS` usando `GIGH_SEQ_LOG_VENTA` mediante su trigger.

Respuesta PL/SQL:

Pregunta 59 — Desafío

Desafío: valide que el precio de venta no sea menor que el precio de compra y que el stock no sea negativo.

Respuesta PL/SQL:

Pregunta 60 — Desafío

Desafío: impida eliminar productos que aparezcan en `GIGH_DETALLE_VENTA` y explique la excepción de integridad referencial.

Respuesta PL/SQL:

Sección 7 — Procedimiento de carga de clientes

Habilidad central: inserción integral, función de correo, concurrencia y manejo de errores.

Pregunta 61 — Afianzamiento

Declare `GIGH_CARGA_CLIENTE` con parámetros para código, nombre, apellidos y correo.

Respuesta PL/SQL:

Pregunta 62 — Afianzamiento

Inserte todos los campos de un cliente y permita que el trigger genere el código cuando sea nulo.

Respuesta PL/SQL:

Pregunta 63 — Afianzamiento

Use `GIGH_GENERA_EMAIL` cuando el correo no sea proporcionado por el llamador.

Respuesta PL/SQL:

Pregunta 64 — Afianzamiento

Permita recibir un correo explícito y compruebe la restricción `UNIQUE`.

Respuesta PL/SQL:

Pregunta 65 — Afianzamiento

Agregue `LOCK TABLE` y confirme una inserción correcta con `COMMIT`.

Respuesta PL/SQL:

Pregunta 66 — Afianzamiento

Capture DUP_VAL_ON_INDEX, VALUE_ERROR y errores inesperados con mensajes claros.

Respuesta PL/SQL:

Pregunta 67 — Desafío

Desafío: cree una excepción propia para nombre o apellido obligatorio y asígnela con PRAGMA EXCEPTION_INIT.

Respuesta PL/SQL:

Pregunta 68 — Desafío

Desafío: demuestre con dos llamadas iguales que la función genera correos diferentes sin intervención manual.

Respuesta PL/SQL:

Pregunta 69 — Desafío

Desafío: diseñe una prueba con `SAVEPOINT` que permita deshacer una carga sin perder una carga anterior válida.

Respuesta PL/SQL:

Pregunta 70 — Desafío

Desafío: documente cómo el bloqueo y la restricción `UNIQUE` trabajan juntos para resolver cargas concurrentes.

Respuesta PL/SQL:

Sección 8 — Secuencia y auditoría de LOG_VENTAS

Habilidad central: secuencia de log, trazabilidad, consultas y auditoría de DML.

Pregunta 71 — Afianzamiento

Cree GIGH_SEQ_LOG_VENTA y documente sus valores inicial, mínimo, máximo y política de ciclo.

Respuesta PL/SQL:

Pregunta 72 — Afianzamiento

Cree el trigger GIGH_LOG_AUTOINCREMENTAL para asignar COD_LOG automáticamente.

Respuesta PL/SQL:

Pregunta 73 — Afianzamiento

Inserte un log con instrucción, detalle y fecha sin indicar el código.

Respuesta PL/SQL:

Pregunta 74 — Afianzamiento

Registre manualmente las operaciones PRODUCTOS_R, PRODUCTOS_U y PRODUCTOS_D.

Respuesta PL/SQL:

Pregunta 75 — Afianzamiento

Consulte el log ordenado por COD_LOG y luego por FECHAHORA.

Respuesta PL/SQL:

Pregunta 76 — Afianzamiento

Muestre cuántas operaciones existen por tipo de instrucción mediante `GROUP BY`.

Respuesta PL/SQL:

Pregunta 77 — Desafío

Desafío: cree un trigger de auditoría de productos para registrar `INSERT`, `UPDATE` y `DELETE` con `USER`.

Respuesta PL/SQL:

Pregunta 78 — Desafío

Desafío: guarde en el detalle de auditoría el código de producto y, en una actualización, los valores anterior y nuevo del stock.

Respuesta PL/SQL:

Pregunta 79 — Desafío

Desafío: pruebe que un ROLLBACK elimina también el registro de auditoría creado dentro de la misma transacción.

Respuesta PL/SQL:

Pregunta 80 — Desafío

Desafío: construya un reporte diario del log con cantidad de operaciones, primera hora y última hora por tipo.

Respuesta PL/SQL:

Lista de comprobación final

- ☐ Todas las tablas tienen el prefijo del estudiante.
- ☐ Los **DROP** respetan las dependencias entre claves foráneas.
- ☐ Los triggers usan correctamente **:NEW** y **:OLD**.
- ☐ Las secuencias se utilizan solo para generar identificadores.
- ☐ Los procedimientos diferencian parámetros y columnas.
- ☐ Cada operación DML tiene tratamiento de transacción.
- ☐ Las excepciones específicas aparecen antes de **WHEN OTHERS**.
- ☐ Se ejecutaron pruebas válidas, inválidas, duplicados y concurrencia.
- ☐ Cada requerimiento está separado con comentarios **--**.